



Reinforcement Learning





iman.mossavat@fontys.nl

- Working for Fontys since August 2022
- Enjoy learning “stuff”



...

If you like this type of material join us at science and math club lunch meetings

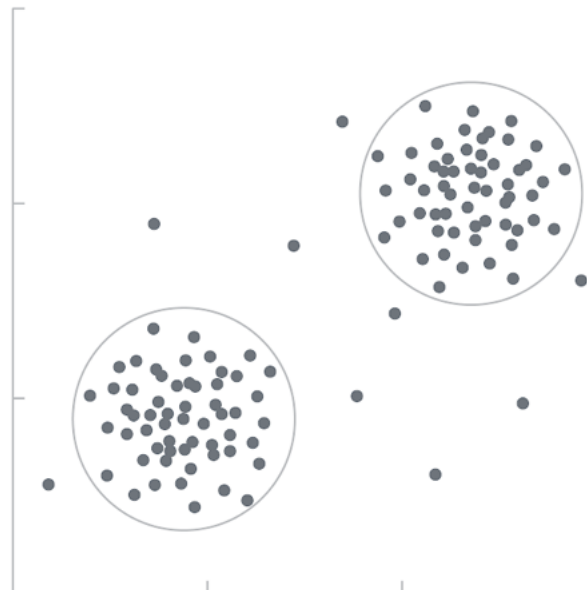
Supervised



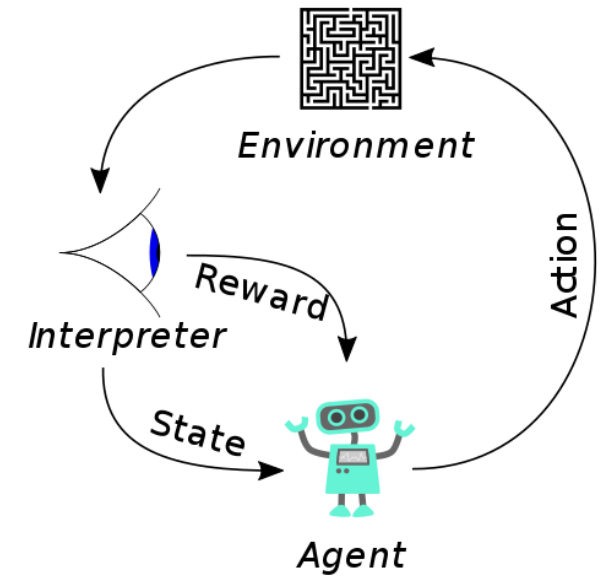
2.jpg

Label: dog

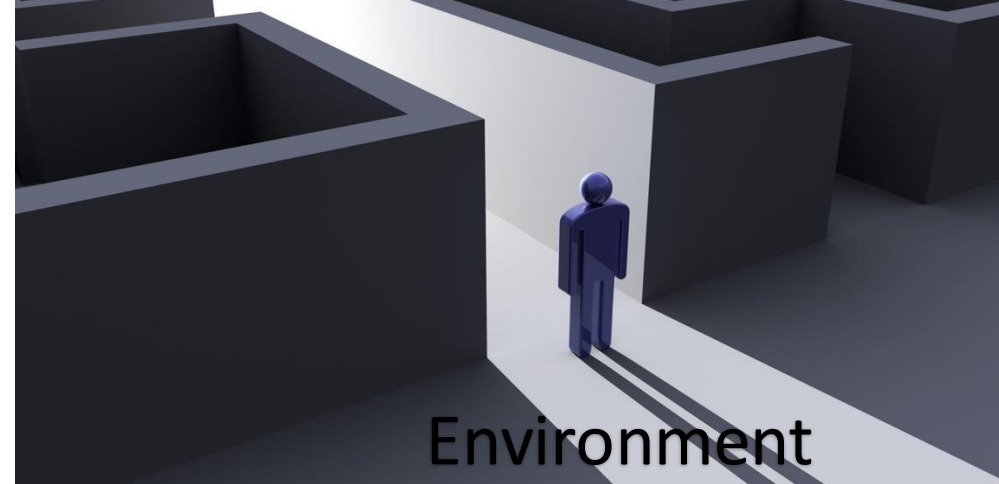
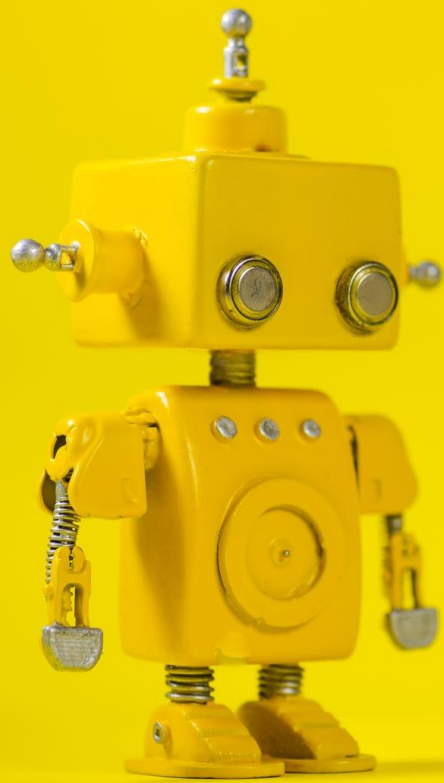
Unsupervised



Reinforcement



Agent (Action holder)



Environment

Uncertainty



Policy



Reward Signal



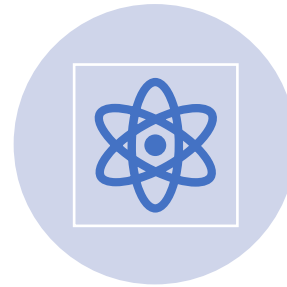
It might
look goofy ...



An Explosion in RL Application



Reinforcement learning from human feedback (RLHF): use in ChatGPT



Accelerating fusion science through learned plasma control



Alpha Tensor: *discovering new algorithms*



References in [RL - Theoretical background](#)

The Learning Loop

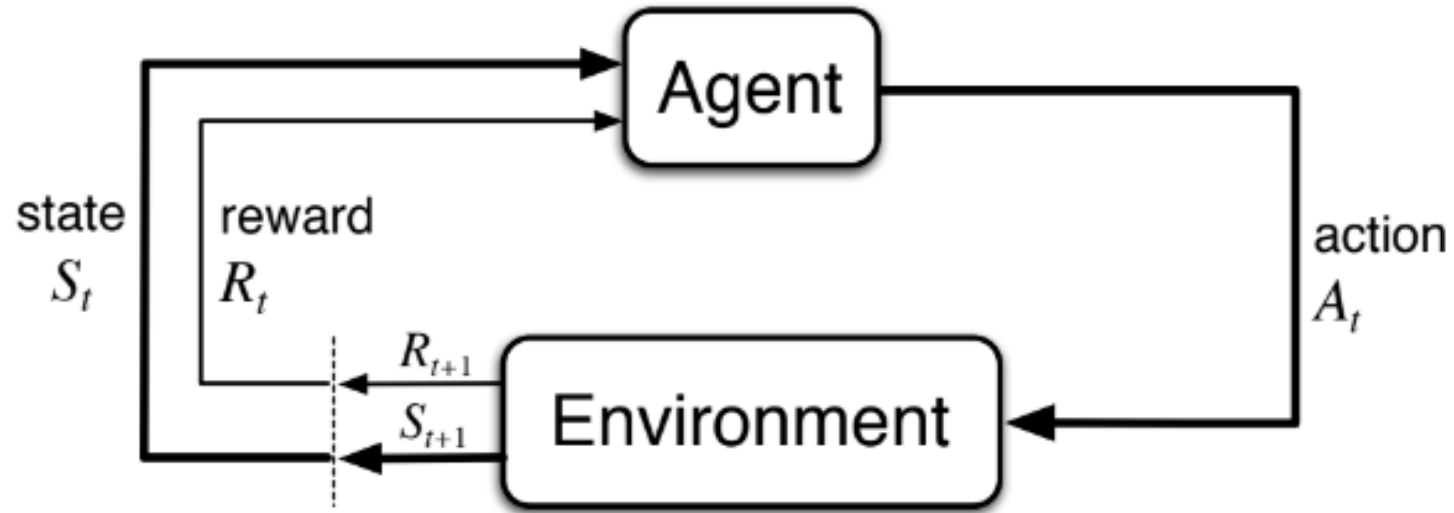


Figure 3.1: The agent–environment interaction in a Markov decision process.

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, \dots, R_T, S_T$$

Expected Future Reward or Quality Q is state and action dependent

If only your agent had a table with all the average reward for each state-action pair (or the quality)

	action 1	action 2	action 3
state 1	0,11	0,51	0,94
state 2	0,09	0,09	0,11
state 3	0,96	0,10	0,30
state 4	0,14	0,21	0,34



The agent does not initially have an overview,
it must experience and find out

In typical scenarios we have to learn the Q function



	action 1	action 2	action 3
state 1	?	?	?
state 2	?	?	?
state 3	?	?	?
state 4	?	?	?

Explore-Exploit Trade-offs



GATHER NEW INFORMATION



LEVERAGING EXISTING
KNOWLEDGE.

Use-cases



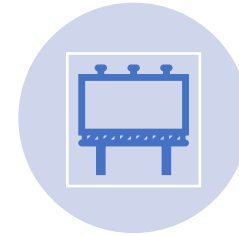
RESTAURANTS: CHOOSING BETWEEN A FAVORITE PLACE AND A NEWLY OPENED RESTAURANT.



MUSIC: OPTING FOR A FAVORITE ALBUM OR EXPLORING NEW MUSIC.



SOCIAL LIVES: BALANCING TIME WITH CLOSE CIRCLE FRIENDS AND FOSTERING NEW ACQUAINTANCESHIPS.



BUSINESS: DECIDING BETWEEN RUNNING A PROVEN AD WITH HIGH CLICK RATES OR TRYING NEW, UNTESTED ADS.



MEDICINE: WEIGHING THE BEST-KNOWN TREATMENT AGAINST EXPERIMENTAL OPTIONS IN CLINICAL TRIALS.



Don't be Greedy

Or you will get stuck

exploration - exploitation



During the learning process, the agent needs to explore different actions and their outcomes to discover the optimal policy.



If the agent only chooses actions randomly, it may take a long time to converge to the optimal policy.



If the agent only chooses actions based on the current Q-values, it may get stuck in a suboptimal policy.



The epsilon-greedy strategy is used to balance between exploration and exploitation by **choosing a random action with a small probability** (epsilon) and choosing the action with the highest Q-value in the current state with probability $(1 - \text{epsilon})$.

Hyper parameters



ϵ (Epsilon, greediness)



η/α (Eta/alpha, learning rate)

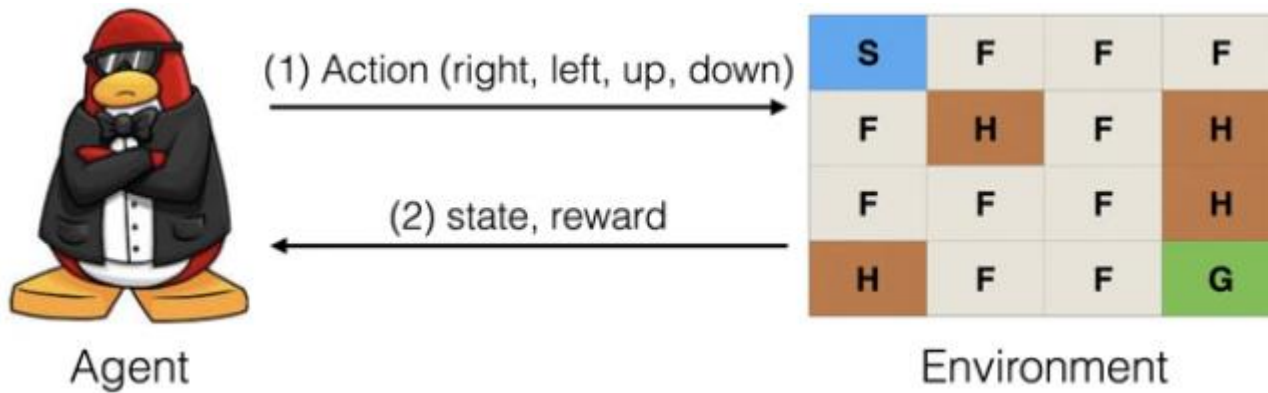


γ (Gamma, discount)

Gymnasium and Tabular Q- Learning Code

Example: Slippery Frozen Lake

[Frozen Lake - Gymnasium Documentation \(farama.org\)](https://farama.org)



The agent starts at **S** and receives a reward when reaching **G**. If it falls into hole **H** the episode ends (and it may start over)

The agent does NOT have this overview initially, it must figure out the map by experimenting.

The environment `env` manages the update of the state and calculation of the reward. It knows in which state we are.

```
import gymnasium as gym
env = gym.make("LunarLander-v2", render_mode="human")
observation, info = env.reset(seed=42)
for _ in range(1000):
    action = env.action_space.sample() # this is where you would insert your policy
    observation, reward, terminated, truncated, info = env.step(action)

    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

[Gymnasium Documentation \(farama.org\)](https://farama.org/)

Initial state

```
import gymnasium as gym
env = gym.make("LunarLander-v2", render_mode="human")
observation, info = env.reset(seed=42)
for _ in range(1000):
    action = env.action_space.sample() # this is where you would insert your policy
    observation, reward, terminated, truncated, info = env.step(action)

    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

[Gymnasium Documentation \(farama.org\)](https://farama.org/)

$T = 1000$ in our slides

```
observation, info = env.reset(seed=42, mode="human")  
  
for _ in range(1000):  
    action = env.action_space.sample() # this is where you would insert your policy  
    observation, reward, terminated, truncated, info = env.step(action)  
  
    if terminated or truncated:  
        observation, info = env.reset()  
  
env.close()
```


`env.action_space` is the set of
feasible actions
(state-dependent)

```
import gymnasium as gym
env = gym.make("LunarLander-v2", render_mode="human")
observation, info = env.reset(seed=42)
for _ in range(1000):
    action = env.action_space.sample() # this is where you would insert your policy
    observation, reward, terminated, truncated, info = env.step(action)

    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

[Gymnasium Documentation \(farama.org\)](https://farama.org/)

Here we select a random move! (random policy, maximum exploration)

```
import gymnasium as gym
env = gym.make("LunarLander-v2", render_mode="human")
observation, info = env.reset(seed=42)
for _ in range(1000):
    action = env.action_space.sample() # this is where you would insert your policy
    observation, reward, terminated, truncated, info = env.step(action)

    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

[Gymnasium Documentation \(farama.org\)](https://farama.org/)

Here we ask the environment to execute the action give the state we are in

```
import gymnasium as gym
env = gym.make("LunarLander-v2", render_mode="human")
observation, info = env.reset(seed=42)
for _ in range(1000):
    action = env.action_space.sample() # this is where you would insert your policy
    observation, reward, terminated, truncated, info = env.step(action)

    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

[Gymnasium Documentation \(farama.org\)](https://farama.org/)

Here we ask the environment to execute the action give the state we are in (state is an environment variable)

```
import gymnasium as gym
env = gym.make("LunarLander-v2", render_mode="human")
observation, info = env.reset(seed=42)
for _ in range(1000):
    action = env.action_space.sample() # this is where you would insert your policy
    observation, reward, terminated, truncated, info = env.step(action)

    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

[Gymnasium Documentation \(farama.org\)](https://farama.org/)

New state

```
import gymnasium as gym
env = gym.make("LunarLander-v2", render_mode="human")
observation, info = env.reset(seed=42)
for _ in range(1000):
    action = env.action_space.sample() # this is where you would insert your policy
    observation, reward, terminated, truncated, info = env.step(action)

    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

[Gymnasium Documentation \(farama.org\)](https://farama.org/)

Tabular Q-Learning Code

- I am going to provide you with a code that can guide your work.
- Try to match the code against the slide contents
- The code will not work as is, you need to figure out how to build the solution yourself, but it should make it clear what I exactly mean

Tabular Q-learning Code: Initialization

```
import numpy as np
```

```
class TabularQLearning:
```

```
    def __init__(self, num_states, num_actions, learning_rate=0.1, discount_factor=0.9, epsilon=0.1):
```

```
        self.num_states = num_states
```

```
        self.num_actions = num_actions
```

```
        self.learning_rate = learning_rate
```

```
        self.discount_factor = discount_factor
```

```
        self.epsilon = epsilon
```

```
        self.q_table = np.zeros((num_states, num_actions))
```


Tabular Q-learning Code: Epsilon Greedy

```
def choose_action(self, state):  
    if np.random.rand() < self.epsilon:  
        return np.random.choice(self.num_actions)  
    else:  
        return np.argmax(self.q_table[state])
```

Tabular Q-learning Code: Q-learning Update

```
def update_q_table(self, state, action, reward, next_state):  
    old_q_value = self.q_table[state, action]  
    next_max = np.max(self.q_table[next_state])  
    new_q_value = old_q_value + self.learning_rate * (reward + self.discount_factor * next_max - old_q_value)  
    self.q_table[state, action] = new_q_value
```

Tabular Q-learning: Main Loop

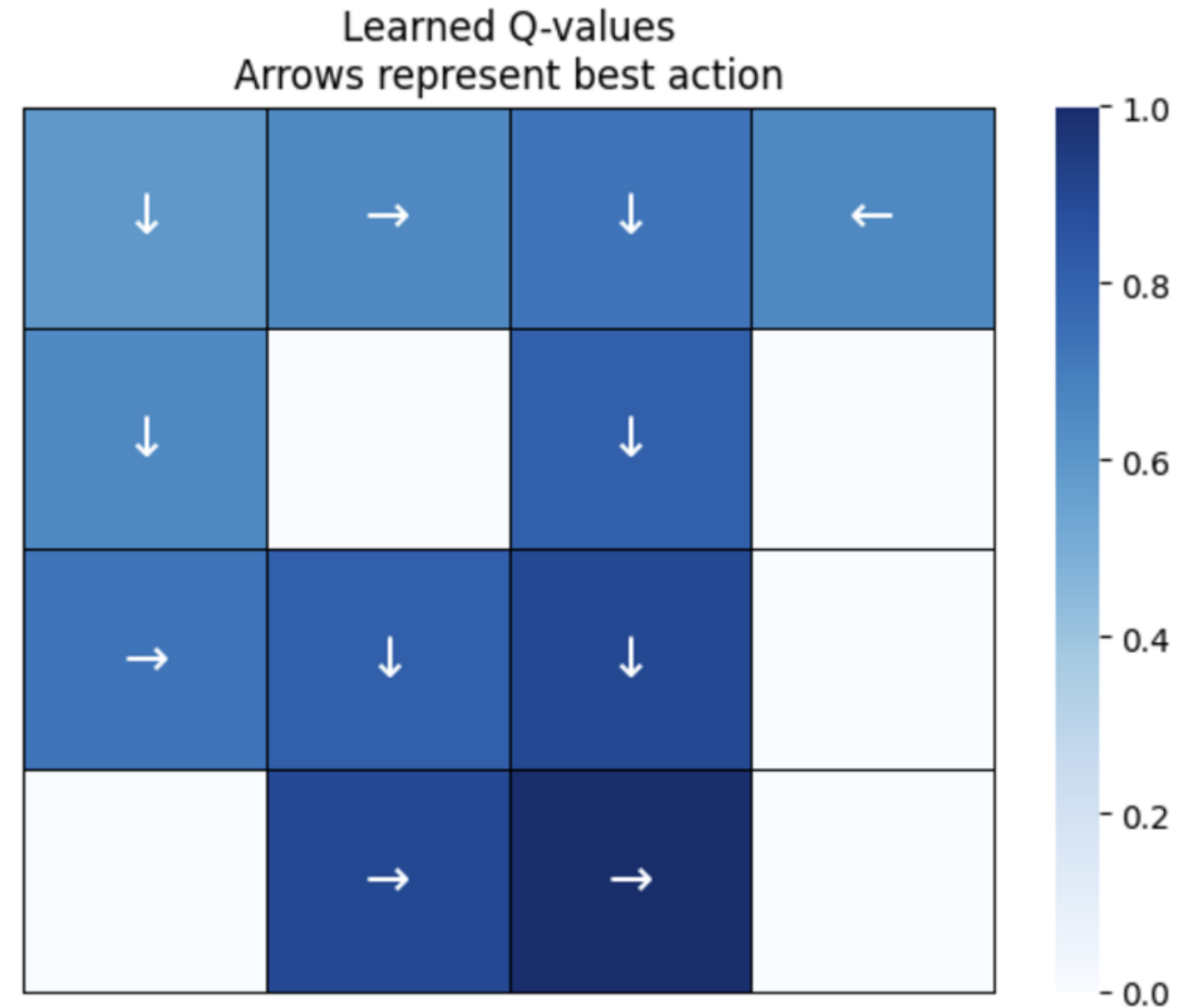
```
# Simulated environment interactions
num_episodes = 1000
for episode in range(num_episodes):
    state = np.random.randint(0, num_states) # Random initial state
    while not done:
        action = agent.choose_action(state)
        # Simulate environment: obtain reward and next state
        next_state, reward, done = env.step(action)
        # Update Q-table based on the observed transition
        agent.update_q_table(state, action, reward, next_state)
        state = next_state
    # YOUR KPI CALCULATIONS HERE TO PROVE AGENT IS LEARNING
```

Assignment

- Any environment from Gymnasium or comparable libraries is acceptable.
- The simplest environment is Frozen Lake, if you want to try it, please use the 8x8 version: [Frozen Lake - Gymnasium Documentation \(farama.org\)](https://gymnasium.farama.org/environments/twenty_four_hours/frozen_lake/)
 - You can solve this with a simple table.
- Try out a deep network without vision with discrete actions
 - (cart-pole)
 - [Classic Control - Gymnasium Documentation \(farama.org\)](https://gymnasium.farama.org/environments/classic_control/)

Generate a Python function that takes a Q-table and map size, returns the maximum Q-values, and the best-learned actions mapped to arrows ($\leftarrow$, $\downarrow$, $\rightarrow$, $\uparrow$) on a grid.

Plot and
justify on
agent
behavior





The End